

```
!pip install torchinfo --quiet
```

```
import time
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split

SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
assert device.type == "cuda", "STOP: you are on CPU. Runtime > Change

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def model_size_mb(model):
    return count_parameters(model) * 4 / (1024 ** 2)

def estimate_recurrent_flops(cell_type, input_size, hidden_size, seq_len,
                             num_layers=1, output_size=None):
    """Analytical forward FLOPs per sequence for comparing cell types.
    Gate multipliers: RNN=1, GRU=3, LSTM=4."""
    gate_mult = {"rnn": 1, "gru": 3, "lstm": 4}[cell_type.lower()]
    flops = 0
    in_sz = input_size
    for layer in range(num_layers):
        per_step = gate_mult * 2 * (in_sz * hidden_size + hidden_size * in_sz)
        flops += per_step * seq_len
        in_sz = hidden_size
    if output_size is not None:
        flops += 2 * hidden_size * output_size
    return flops
```

Using device: cuda

```

import requests

url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tiny_shakespeare.txt"
shakespeare_text = requests.get(url).text
print("Tiny Shakespeare length (chars):", len(shakespeare_text))

shake_chars = sorted(list(set(shakespeare_text)))
shake_vocab = len(shake_chars)
shake_char_to_int = {ch: i for i, ch in enumerate(shake_chars)}
shake_int_to_char = {i: ch for i, ch in enumerate(shake_chars)}
print("Vocabulary size:", shake_vocab)

encoded_shake = [shake_char_to_int[ch] for ch in shakespeare_text]

```

```

Tiny Shakespeare length (chars): 1115394
Vocabulary size: 65

```

```

class ShakespeareDataset(Dataset):
    def __init__(self, encoded_text, sequence_length):
        self.seq_len = sequence_length
        seqs, targs = [], []
        for i in range(0, len(encoded_text) - sequence_length):
            seqs.append(encoded_text[i:i + sequence_length])
            targs.append(encoded_text[i + sequence_length])
        self.sequences = torch.tensor(seqs, dtype=torch.long)
        self.targets = torch.tensor(targs, dtype=torch.long)

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        return self.sequences[idx], self.targets[idx]

def make_shakespeare_loaders(encoded_text, sequence_length, batch_size,
                             subset=None):
    """Set subset=None for the full dataset in your final run."""
    data = encoded_text if subset is None else encoded_text[:subset]
    dataset = ShakespeareDataset(data, sequence_length)
    train_size = int(len(dataset) * 0.8)
    test_size = len(dataset) - train_size
    train_ds, test_ds = random_split(
        dataset, [train_size, test_size],
        generator=torch.Generator().manual_seed(SEED))

```

```

train_loader = DataLoader(train_ds, shuffle=True, batch_size=batch_size)
test_loader = DataLoader(test_ds, shuffle=False, batch_size=batch_size)
return train_loader, test_loader

class ShakespeareModel(nn.Module):
    """Embedding -> (LSTM/GRU, multi-layer) -> optional extra FC layer
    def __init__(self, vocab_size, embed_size, hidden_size, output_size,
                  cell_type="lstm", num_layers=1, fc_layers=1, dropout=0.0):
        super().__init__()
        self.cell_type = cell_type.lower()
        self.num_layers = num_layers
        self.hidden_size = hidden_size
        self.embedding = nn.Embedding(vocab_size, embed_size)
        rnn_cls = {"lstm": nn.LSTM, "gru": nn.GRU}[self.cell_type]
        self.rnn = rnn_cls(embed_size, hidden_size, num_layers=num_layers,
                           batch_first=True,
                           dropout=dropout if num_layers > 1 else 0.0)
        layers = []
        in_features = hidden_size
        for _ in range(max(fc_layers - 1, 0)):
            layers += [nn.Linear(in_features, hidden_size), nn.ReLU()]
            in_features = hidden_size
        layers.append(nn.Linear(in_features, output_size))
        self.fc = nn.Sequential(*layers)

    def forward(self, x):
        embedded = self.embedding(x)
        output, _ = self.rnn(embedded)
        return self.fc(output[:, -1, :])

def train_eval_loader(model, train_loader, test_loader, epochs=5, lr=0.001,
                      clip_norm=5.0, label="", log_every=1):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_loss_hist, val_loss_hist, val_acc_hist = [], [], []
    start = time.time()
    for epoch in range(epochs):
        model.train()
        running, seen = 0.0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)

```

```

        optimizer.zero_grad()
        out = model(xb)
        loss = criterion(out, yb)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm)
        optimizer.step()
        running += loss.item() * xb.size(0)
        seen += xb.size(0)
    train_loss = running / seen

    model.eval()
    v_loss, v_correct, v_seen = 0.0, 0, 0
    inf_start = time.time()
    with torch.no_grad():
        for xb, yb in test_loader:
            xb, yb = xb.to(device), yb.to(device)
            out = model(xb)
            v_loss += criterion(out, yb).item() * xb.size(0)
            v_correct += (out.argmax(1) == yb).sum().item()
            v_seen += xb.size(0)
    inf_time = time.time() - inf_start
    val_loss = v_loss / v_seen
    val_acc = v_correct / v_seen

    train_loss_hist.append(train_loss)
    val_loss_hist.append(val_loss)
    val_acc_hist.append(val_acc)
    if (epoch + 1) % log_every == 0:
        print(f"[{label}] Epoch {epoch+1}/{epochs}  train_loss={train_loss:.4f}  "
              f"val_loss={val_loss:.4f}  val_acc={val_acc:.4f}")

    train_time = time.time() - start
    perplexity = math.exp(val_loss_hist[-1])
    n_val_batches = max(len(test_loader), 1)
    return {
        "train_loss": train_loss_hist,
        "val_loss": val_loss_hist,
        "val_acc": val_acc_hist,
        "final_train_loss": train_loss_hist[-1],
        "final_val_loss": val_loss_hist[-1],
        "final_val_acc": val_acc_hist[-1],
        "best_val_acc": max(val_acc_hist),
        "train_time_s": train_time,
        "inference_time_per_batch_ms": (inf_time / n_val_batches) * 1000,
        "perplexity": perplexity,
    }

```

```

    }

def generate_text(model, seed_str, length, char_to_int, int_to_char, device):
    model.eval()
    generated = seed_str
    with torch.no_grad():
        for _ in range(length):
            context = generated[-seq_len:]
            idx = [char_to_int.get(c, 0) for c in context]
            x = torch.tensor([idx], dtype=torch.long).to(device)
            out = model(x)
            nxt = int_to_char[out.argmax(1).item()]
            generated += nxt
    return generated

```

```

SUBSET = 200000
EPOCHS_P2 = 5
EMBED_P2 = 64
HIDDEN_P2 = 256
BATCH_P2 = 128

seq_lengths_p2 = [20, 30]
cells_p2 = ["lstm", "gru"]

p2_results = {}
p2_rows = []

for seq_len in seq_lengths_p2:
    train_loader, test_loader = make_shakespeare_loaders(
        encoded_shake, seq_len, batch_size=BATCH_P2, subset=SUBSET)
    for cell in cells_p2:
        torch.manual_seed(SEED)
        model = ShakespeareModel(shake_vocab, EMBED_P2, HIDDEN_P2, shake_vocab,
                                cell_type=cell, num_layers=1, fc_layer=1)
        label = f"{cell.upper()}-L{seq_len}"
        hist = train_eval_loader(model, train_loader, test_loader,
                                epochs=EPOCHS_P2, label=label)
        flops = estimate_recurrent_flops(cell, EMBED_P2, HIDDEN_P2, seq_len,
                                         num_layers=1, output_size=shake_vocab.get_vocab_size('tokens'))
        p2_results[(cell, seq_len)] = hist
        p2_rows.append({
            "Model": cell.upper(),
            "Seq Len": seq_len,
            "Flops": flops,
            "Train Loss": hist["train_loss"],
            "Train Acc": hist["train_acc"],
            "Test Loss": hist["test_loss"],
            "Test Acc": hist["test_acc"]
        })

```

```

    final_train_loss : round(hist["final_train_loss"], 4),
    "Final Val Loss": round(hist["final_val_loss"], 4),
    "Final Val Acc": round(hist["final_val_acc"], 4),
    "Perplexity": round(hist["perplexity"], 3),
    "Train Time (s)": round(hist["train_time_s"], 2),
    "Inf Time/batch (ms)": round(hist["inference_time_per_batch"], 2),
    "Params": count_parameters(model),
    "Size (MB)": round(model_size_mb(model), 3),
    "FLOPs/seq": flops,
})

```

```

p2_df = pd.DataFrame(p2_rows)
p2_df

```

```

[LSTM-L20] Epoch 1/5  train_loss=1.8890  val_loss=1.6366  val_acc=0.511
[LSTM-L20] Epoch 2/5  train_loss=1.5542  val_loss=1.5423  val_acc=0.533
[LSTM-L20] Epoch 3/5  train_loss=1.4577  val_loss=1.5099  val_acc=0.539
[LSTM-L20] Epoch 4/5  train_loss=1.3959  val_loss=1.4856  val_acc=0.550
[LSTM-L20] Epoch 5/5  train_loss=1.3493  val_loss=1.4951  val_acc=0.548
[GRU-L20] Epoch 1/5  train_loss=1.8820  val_loss=1.6776  val_acc=0.5031
[GRU-L20] Epoch 2/5  train_loss=1.6083  val_loss=1.6202  val_acc=0.5164
[GRU-L20] Epoch 3/5  train_loss=1.5386  val_loss=1.5980  val_acc=0.5269
[GRU-L20] Epoch 4/5  train_loss=1.5082  val_loss=1.5962  val_acc=0.5274
[GRU-L20] Epoch 5/5  train_loss=1.4935  val_loss=1.6183  val_acc=0.5286
[LSTM-L30] Epoch 1/5  train_loss=1.8740  val_loss=1.6235  val_acc=0.514
[LSTM-L30] Epoch 2/5  train_loss=1.5426  val_loss=1.5315  val_acc=0.538
[LSTM-L30] Epoch 3/5  train_loss=1.4489  val_loss=1.5010  val_acc=0.544
[LSTM-L30] Epoch 4/5  train_loss=1.3907  val_loss=1.4802  val_acc=0.549
[LSTM-L30] Epoch 5/5  train_loss=1.3457  val_loss=1.4759  val_acc=0.555
[GRU-L30] Epoch 1/5  train_loss=1.8737  val_loss=1.6605  val_acc=0.5049
[GRU-L30] Epoch 2/5  train_loss=1.6041  val_loss=1.6207  val_acc=0.5195
[GRU-L30] Epoch 3/5  train_loss=1.5359  val_loss=1.5914  val_acc=0.5239
[GRU-L30] Epoch 4/5  train_loss=1.5065  val_loss=1.5868  val_acc=0.5268
[GRU-L30] Epoch 5/5  train_loss=1.4940  val_loss=1.5868  val_acc=0.5305

```

	Model	Seq Len	Final Train Loss	Final Val Loss	Final Val Acc	Perplexity	Train Time (s)	Inf Time/batch (ms)	Params
0	LSTM	20	1.3493	1.4951	0.5480	4.460	27.83	1.744	350593
1	GRU	20	1.4935	1.6183	0.5286	5.044	25.82	1.718	268161
2	LSTM	30	1.3457	1.4759	0.5550	4.375	34.03	2.164	350593

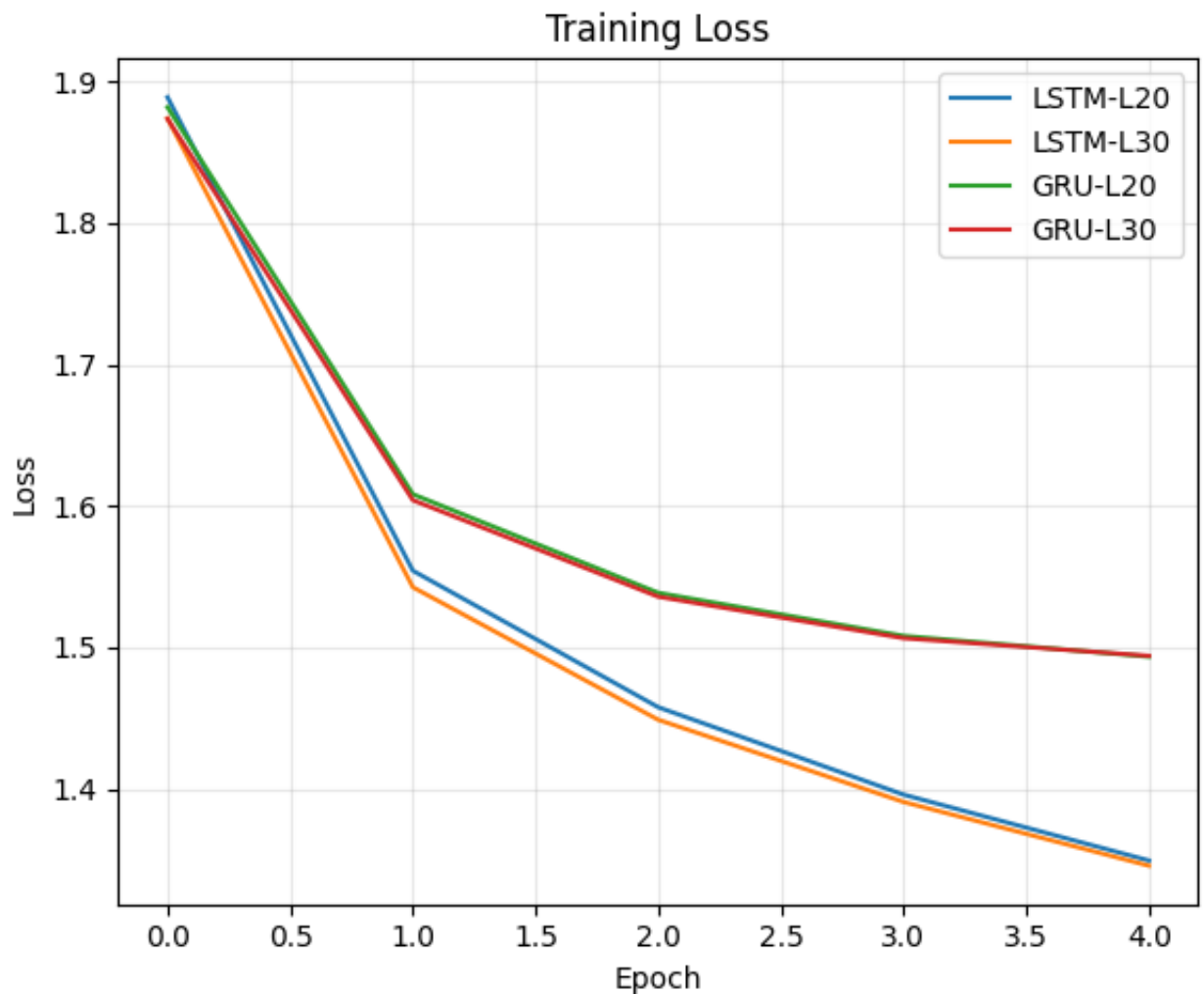
Next steps:

[Generate code with p2_df](#)[New interactive sheet](#)

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for cell in cells_p2:
    for seq_len in seq_lengths_p2:
        tag = f"{cell.upper()}-L{seq_len}"
        h = p2_results[(cell, seq_len)]
        axes[0].plot(h["train_loss"], label=tag)
        axes[1].plot(h["val_loss"], label=tag)
        axes[2].plot(h["val_acc"], label=tag)
axes[0].set_title("Training Loss"); axes[0].set_xlabel("Epoch"); axes
axes[1].set_title("Validation Loss"); axes[1].set_xlabel("Epoch"); axes
axes[2].set_title("Validation Accuracy"); axes[2].set_xlabel("Epoch");
for ax in axes:
    ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```




```

SEQ_LEN_HP = 30
EPOCHS_HP = 5
hp_configs = [
    {"name": "baseline",      "cell": "lstm", "hidden": 256, "num_layer
    {"name": "wide_hidden",   "cell": "lstm", "hidden": 512, "num_layer
    {"name": "deep_recurrent", "cell": "lstm", "hidden": 256, "num_layer
    {"name": "deep_fc",        "cell": "lstm", "hidden": 256, "num_layer
    {"name": "gru_baseline",   "cell": "gru",  "hidden": 256, "num_layer
]

train_loader_hp, test_loader_hp = make_shakespeare_loaders(
    encoded_shake, SEQ_LEN_HP, batch_size=BATCH_P2, subset=SUBSET)

hp_rows = []
hp_results = {}
for cfg in hp_configs:
    torch.manual_seed(SEED)
    model = ShakespeareModel(shake_vocab, EMBED_P2, cfg["hidden"], shak
                           cell_type=cfg["cell"], num_layers=cfg["num
                           fc_layers=cfg["fc_layers"],
                           dropout=0.2 if cfg["num_layers"] > 1 else
    hist = train_eval_loader(model, train_loader_hp, test_loader_hp,
                             epochs=EPOCHS_HP, label=cfg["name"])
    sample = generate_text(model, "ROMEO: ", 200, shake_char_to_int,
                           shake_int_to_char, SEQ_LEN_HP)
    flops = estimate_recurrent_flops(cfg["cell"], EMBED_P2, cfg["hidden
                                   SEQ_LEN_HP, num_layers=cfg["num_la
                                   output_size=shake_vocab)
    hp_results[cfg["name"]] = {"hist": hist, "sample": sample}
    hp_rows.append({
        "Config": cfg["name"],
        "Cell": cfg["cell"].upper(),
        "Hidden": cfg["hidden"],
        "Rec Layers": cfg["num_layers"],
        "FC Layers": cfg["fc_layers"],
        "Val Loss": round(hist["final_val_loss"], 4),
        "Val Acc": round(hist["final_val_acc"], 4),
        "Perplexity": round(hist["perplexity"], 3),
        "Train Time (s)": round(hist["train_time_s"], 2),
        "Inf/batch (ms)": round(hist["inference_time_per_batch_ms"], 3)
        "Params": count_parameters(model),
        "Size (MB)": round(model_size_mb(model), 3),
        "FLOPs/seq": flops,
    })

```

```
hp_df = pd.DataFrame(hp_rows)
hp_df
```

```
[baseline] Epoch 1/5  train_loss=1.8730  val_loss=1.6292  val_acc=0.512
[baseline] Epoch 2/5  train_loss=1.5436  val_loss=1.5351  val_acc=0.538
[baseline] Epoch 3/5  train_loss=1.4492  val_loss=1.4966  val_acc=0.549
[baseline] Epoch 4/5  train_loss=1.3892  val_loss=1.4807  val_acc=0.550
[baseline] Epoch 5/5  train_loss=1.3447  val_loss=1.4807  val_acc=0.552
[wide_hidden] Epoch 1/5  train_loss=1.8347  val_loss=1.5904  val_acc=0.
[wide_hidden] Epoch 2/5  train_loss=1.5012  val_loss=1.4924  val_acc=0.
[wide_hidden] Epoch 3/5  train_loss=1.4006  val_loss=1.4717  val_acc=0.
[wide_hidden] Epoch 4/5  train_loss=1.3259  val_loss=1.4526  val_acc=0.
[wide_hidden] Epoch 5/5  train_loss=1.2724  val_loss=1.4630  val_acc=0.
[deep_recurrent] Epoch 1/5  train_loss=1.9209  val_loss=1.6196  val_acc
[deep_recurrent] Epoch 2/5  train_loss=1.5597  val_loss=1.5099  val_acc
[deep_recurrent] Epoch 3/5  train_loss=1.4679  val_loss=1.4823  val_acc
[deep_recurrent] Epoch 4/5  train_loss=1.4182  val_loss=1.4599  val_acc
[deep_recurrent] Epoch 5/5  train_loss=1.3822  val_loss=1.4526  val_acc
[deep_fc] Epoch 1/5  train_loss=1.8551  val_loss=1.6113  val_acc=0.5080
[deep_fc] Epoch 2/5  train_loss=1.5278  val_loss=1.5309  val_acc=0.5310
[deep_fc] Epoch 3/5  train_loss=1.4347  val_loss=1.4779  val_acc=0.5526
[deep_fc] Epoch 4/5  train_loss=1.3744  val_loss=1.4751  val_acc=0.5535
[deep_fc] Epoch 5/5  train_loss=1.3230  val_loss=1.4714  val_acc=0.5582
[gru_baseline] Epoch 1/5  train_loss=1.8739  val_loss=1.6621  val_acc=0
[gru_baseline] Epoch 2/5  train_loss=1.6042  val_loss=1.6154  val_acc=0
[gru_baseline] Epoch 3/5  train_loss=1.5381  val_loss=1.5959  val_acc=0
[gru_baseline] Epoch 4/5  train_loss=1.5104  val_loss=1.5935  val_acc=0
[gru_baseline] Epoch 5/5  train_loss=1.4915  val_loss=1.5950  val_acc=0
```

	Config	Cell	Hidden	Rec Layers	FC Layers	Val Loss	Val Acc	Perplexity
0	baseline	LSTM	256	1	1	1.4807	0.5523	4.396
1	wide_hidden	LSTM	512	1	1	1.4630	0.5580	4.319
2	deep_recurrent	LSTM	256	2	1	1.4526	0.5582	4.274
3	deep_fc	LSTM	256	1	2	1.4714	0.5582	4.355

Next steps:

[Generate code with hp_df](#)[New interactive sheet](#)

```
for name, r in hp_results.items():
    print(f"\n===== {name} =====")
    print(r["sample"])
```

```
===== baseline =====
```

ROMEO: we will not so die the people,
 And the people are a grave of the people,
 And the people are a grave of the people,
 And the people are a grave of the people,
 And the people are a grave of the people,
 A

===== wide_hidden =====

ROMEO: I have been man he has the people,
 And the people and the people,
 And the people and the people,
 And the people and the people,
 And the people and the people,
 And the people and the people,
 And the people and the people,
 And the pe

===== deep_recurrent =====

ROMEO: I have not the people,
 The people and the people and the people
 The people and the people and the people
 The people and the people and the people
 The people and the people and the people
 The people an

===== deep_fc =====

ROMEO: all the people,
 And the people, and the people,
 And the people, and the people,
 And the people, and the people,
 And the people, and the people,
 And the people, and the people,
 And the people, and the

===== gru_baseline =====

ROMEO: have but the commons.

GLOUCESTE:

Why, I will be with the blood more but the commons.

GLOUCESTE:

Why, I will be with the blood more but the commons.

GLOUCESTE:

Why, I will be with the blood more but

```
SEQ_LEN_50 = 50
```

```
EPOCHS_P23 = 5
```

```
p23_rows = []
```

```

p23_results = {}
train_loader_50, test_loader_50 = make_shakespeare_loaders(
    encoded_shake, SEQ_LEN_50, batch_size=BATCH_P2, subset=SUBSET)

for cell in cells_p2:
    torch.manual_seed(SEED)
    model = ShakespeareModel(shake_vocab, EMBED_P2, HIDDEN_P2, shake_vocab,
                             cell_type=cell, num_layers=1, fc_layers=1)
    hist = train_eval_loader(model, train_loader_50, test_loader_50,
                             epochs=EPOCHS_P23, label=f"{cell.upper()}")
    flops = estimate_recurrent_flops(cell, EMBED_P2, HIDDEN_P2, SEQ_LEN_50,
                                     num_layers=1, output_size=shake_vocab.get_vocab_size('tokens'))

    p23_results[cell] = hist
    p23_rows.append({
        "Model": cell.upper(),
        "Seq Len": 50,
        "Final Val Loss": round(hist["final_val_loss"], 4),
        "Final Val Acc": round(hist["final_val_acc"], 4),
        "Perplexity": round(hist["perplexity"], 3),
        "Train Time (s)": round(hist["train_time_s"], 2),
        "Params": count_parameters(model),
        "Size (MB)": round(model_size_mb(model), 3),
        "FLOPs/seq": flops,
    })

p23_df = pd.DataFrame(p23_rows)
assert len(p23_df) == 2, f"Expected 2 rows (LSTM + GRU), got {len(p23_df)}"
print(p23_df.to_string(index=False))
p23_df

```

```

[LSTM-L50] Epoch 1/5  train_loss=1.8826  val_loss=1.6496  val_acc=0.503
[LSTM-L50] Epoch 2/5  train_loss=1.5509  val_loss=1.5422  val_acc=0.533
[LSTM-L50] Epoch 3/5  train_loss=1.4540  val_loss=1.5129  val_acc=0.544
[LSTM-L50] Epoch 4/5  train_loss=1.3926  val_loss=1.4821  val_acc=0.553
[LSTM-L50] Epoch 5/5  train_loss=1.3453  val_loss=1.4807  val_acc=0.550
[GRU-L50] Epoch 1/5  train_loss=1.8741  val_loss=1.6712  val_acc=0.5064
[GRU-L50] Epoch 2/5  train_loss=1.5968  val_loss=1.6067  val_acc=0.5239
[GRU-L50] Epoch 3/5  train_loss=1.5343  val_loss=1.5967  val_acc=0.5241
[GRU-L50] Epoch 4/5  train_loss=1.5027  val_loss=1.5843  val_acc=0.5317
[GRU-L50] Epoch 5/5  train_loss=1.4880  val_loss=1.6018  val_acc=0.5291
Model  Seq Len  Final Val Loss  Final Val Acc  Perplexity  Train Time (s)
LSTM      50      1.4807      0.5508      4.396      50.
GRU       50      1.6018      0.5291      4.962      45.

```

Model	Seq Len	Final Val Loss	Final Val Acc	Perplexity	Train Time (s)	Params	Size (MB)	FLOPs/seq
LSTM	50	1.4807	0.5508	4.396	50.			
GRU	50	1.6018	0.5291	4.962	45.			

0 LSTM 50 1.4807 0.5508 4.396 50.20 350593 1.337 32801280

Next steps: [Generate code with p23_df](#) [New interactive sheet](#)

```
trend_rows = []
for cell in cells_p2:
    for seq_len in [20, 30]:
        h = p2_results[(cell, seq_len)]
        trend_rows.append({"Model": cell.upper(), "Seq Len": seq_len,
                           "Val Acc": round(h["final_val_acc"], 4),
                           "Perplexity": round(h["perplexity"], 3)})
    h = p23_results[cell]
    trend_rows.append({"Model": cell.upper(), "Seq Len": 50,
                       "Val Acc": round(h["final_val_acc"], 4),
                       "Perplexity": round(h["perplexity"], 3)})

trend_df = pd.DataFrame(trend_rows).sort_values(["Model", "Seq Len"])
assert len(trend_df) == 6, f"Expected 6 rows (2 cells x 3 lengths), got {len(trend_df)}"

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for cell in cells_p2:
    sub = trend_df[trend_df["Model"] == cell.upper()].sort_values("Seq Len")
    axes[0].plot(sub["Seq Len"], sub["Val Acc"], marker="o", label=cell)
    axes[1].plot(sub["Seq Len"], sub["Perplexity"], marker="o", label=cell)
axes[0].set_title("Validation Accuracy vs Sequence Length")
axes[0].set_xlabel("Sequence Length"); axes[0].set_ylabel("Accuracy")
axes[0].set_xticks([20, 30, 50]); axes[0].legend(); axes[0].grid(True)
axes[1].set_title("Perplexity vs Sequence Length")
axes[1].set_xlabel("Sequence Length"); axes[1].set_ylabel("Perplexity")
axes[1].set_xticks([20, 30, 50]); axes[1].legend(); axes[1].grid(True)
plt.tight_layout()
plt.show()

print(trend_df.to_string(index=False))
trend_df
```

